Résolution du problème à N-corps

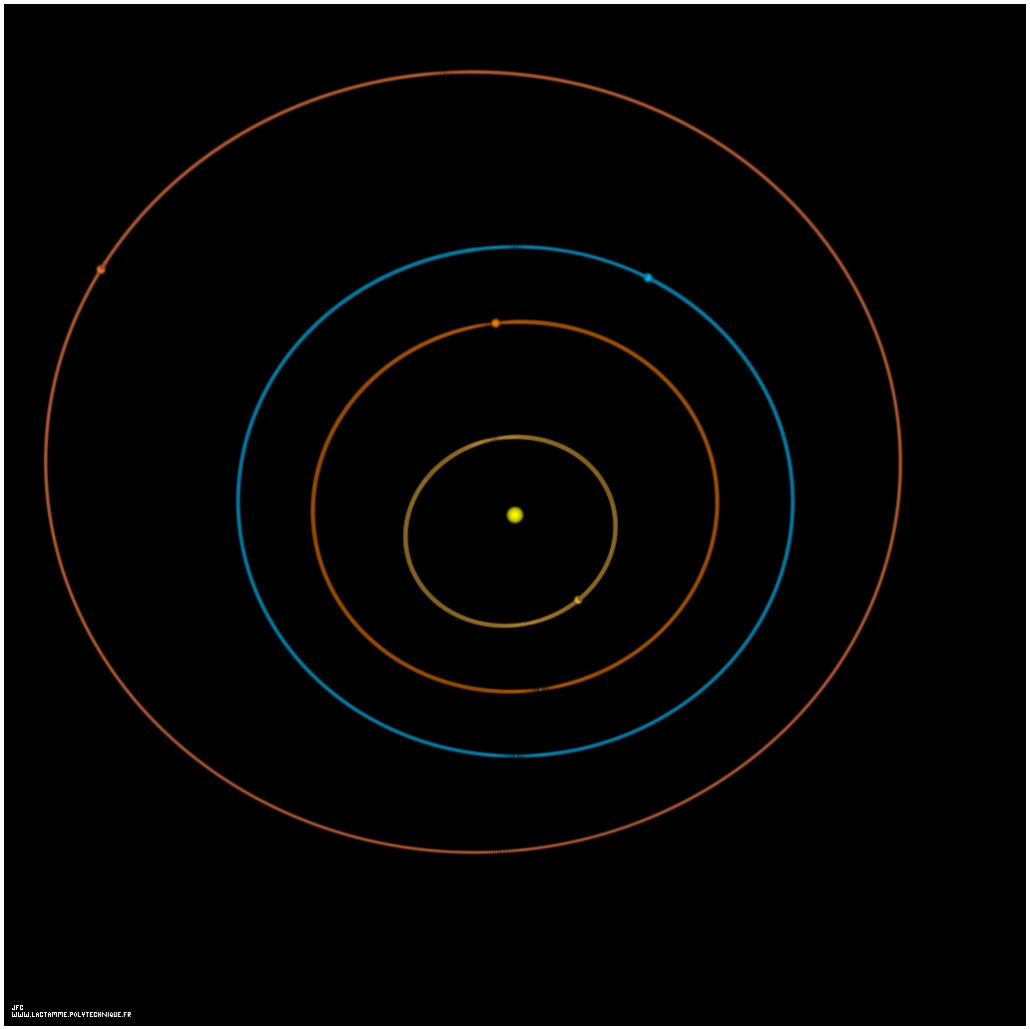
15171 MACCALLUM Jake

2026

Sommaire

- Introduction
- Idée centrale
- Barnes-Hut
- FMM (Fast Multipole Method)
- Résultats
- Annexes

Introduction

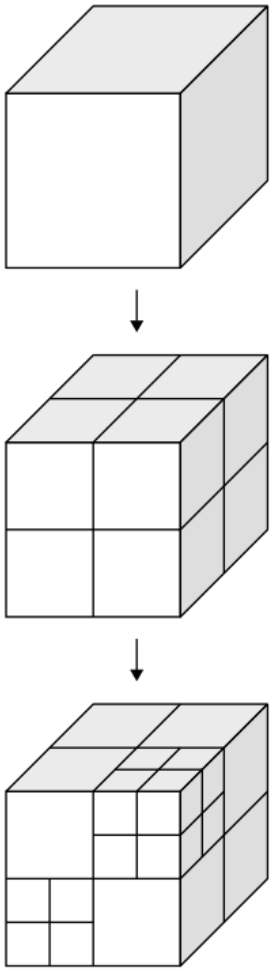


N-Body Problem

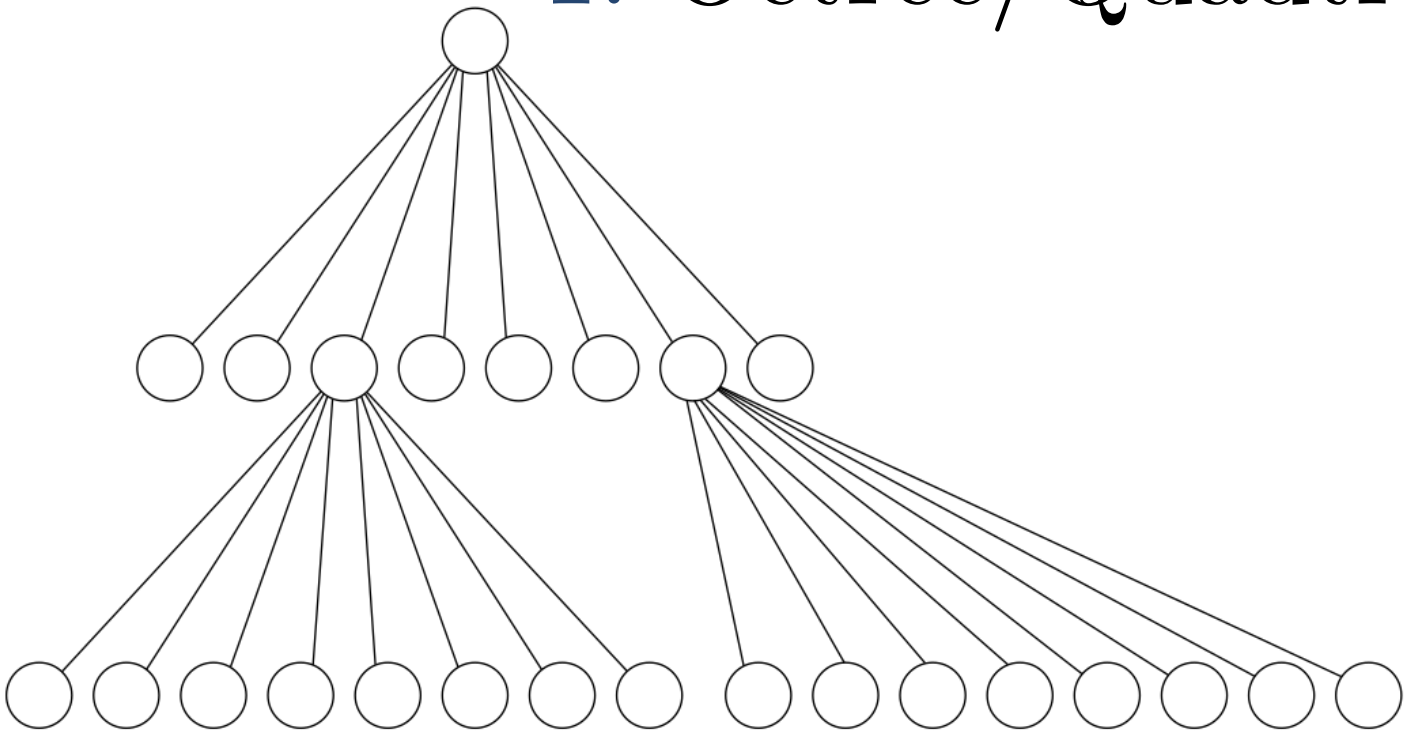
$$\Psi(X_j) = \sum_{\substack{i=1 \\ i \neq j}}^{N} \frac{q_i}{|X_i - X_j|}$$

Compléxité naïve : $O(N^2)$

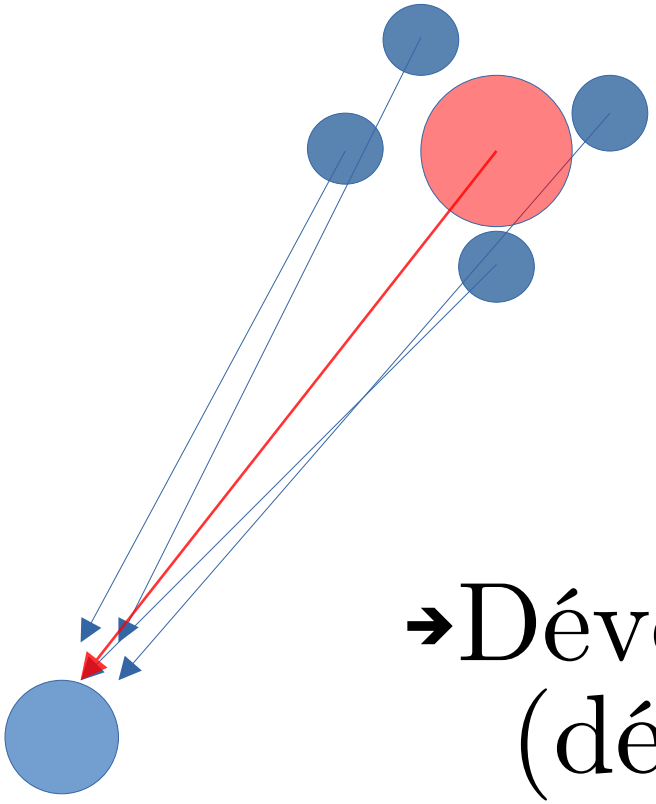
Idée Centrale



1. Octree/Quadtree



Idée Centrale



2. Approximation

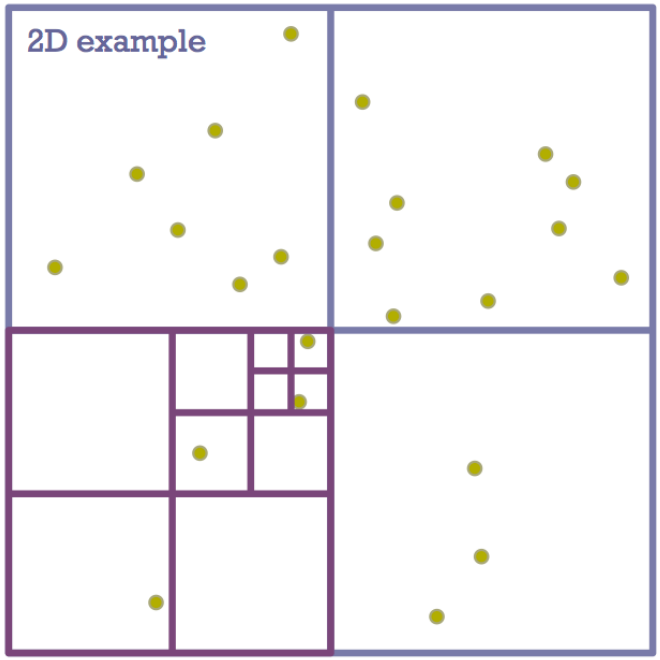
➔Barycentre
(masse et position)

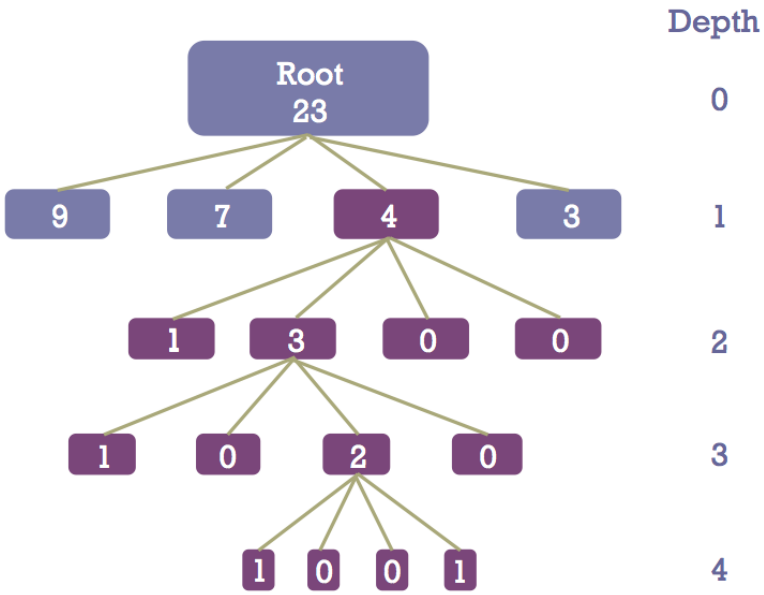
➔Développement multipolaire
(développement en série)

Barnes-Hut

$$\theta = \frac{\text{taille node}}{\text{distance}}$$

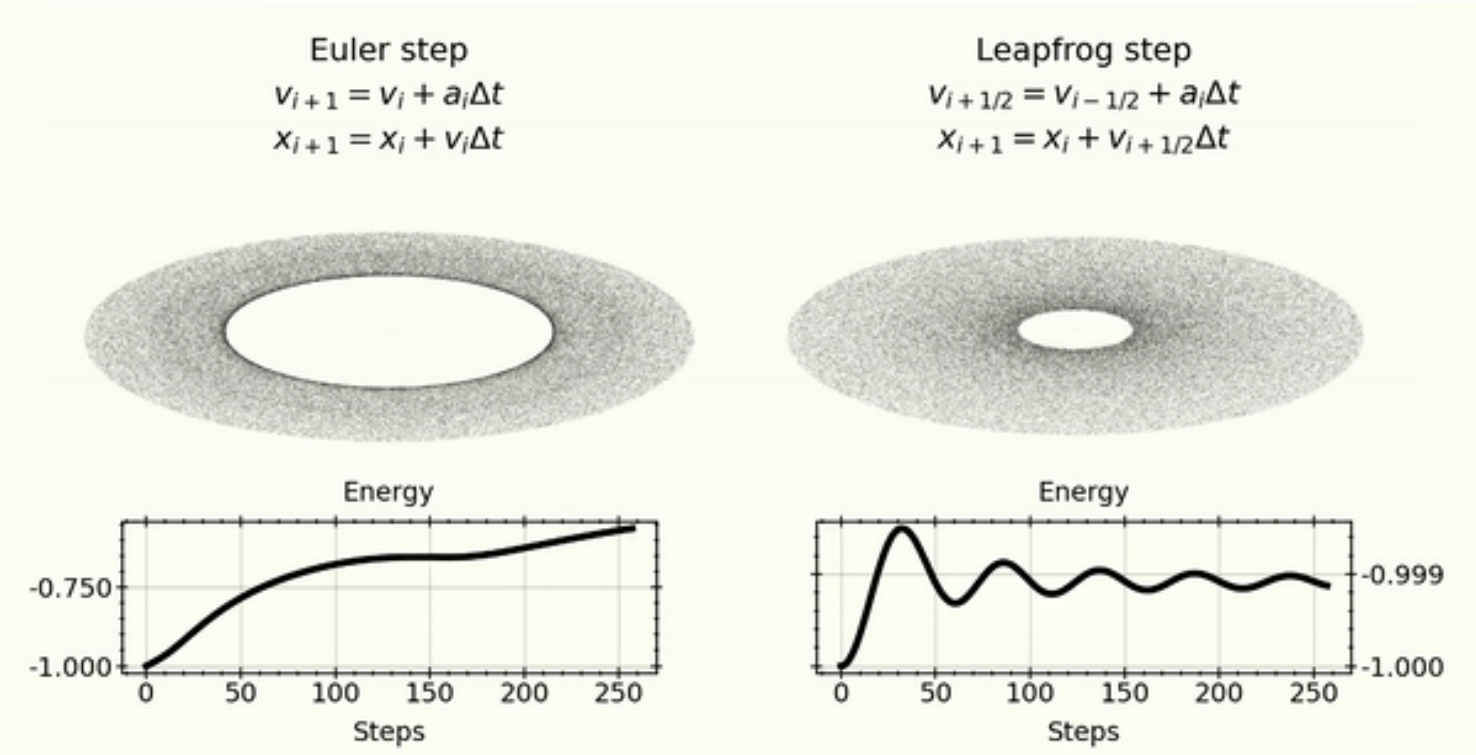
$$O(N \log(N))$$





Erreur de la Terre après 1 jour : 219.559km
(Par rapport à Horizons)

Leapfrog integration



FMM

$$\Phi(X) = \sum_{n=0}^{\infty} \sum_{m=-n}^{n} \frac{M_n^m}{r^{n+1}} \cdot Y_n^m(\theta, \phi),$$

$$O(N) + C$$

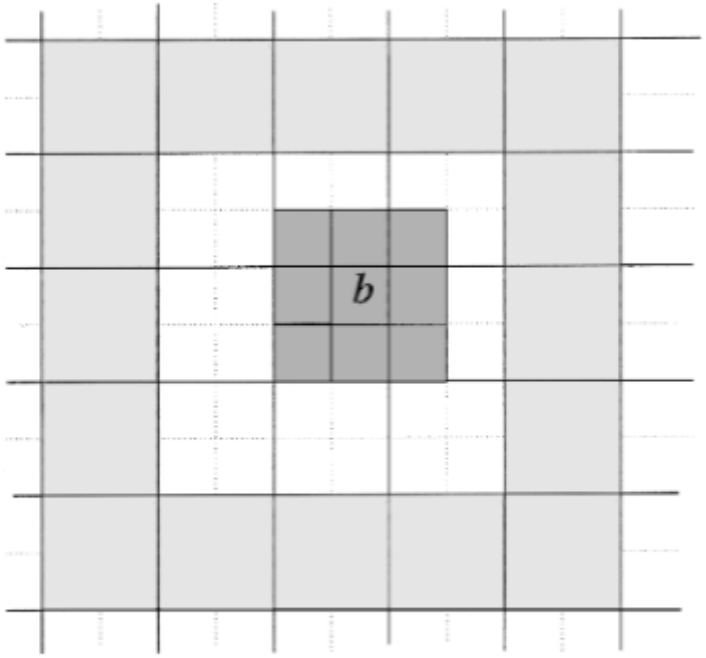


FIG. 1. The colleagues of a (two-dimensional) box *b* are darkly shaded, while its interaction list is indicated in white. In three dimensions, a box *b* has up to 27 colleagues and its interaction list contains up to 189 boxes.

Ordre $p \Rightarrow (4p)!$

$20! = 2.10^{18}$

$\text{Max\_Int\_64} = 9.10^{18}$

Barnes-Hut vs FMM

	Barnes-Hut	FMM
Méthode d'approximation	Barycentre	Moments multipolaires
Erreur	$\theta = \frac{\text{taille node}}{\text{distance}}$	Ordre du DL
Complexité	$O(n \log(n))$	$O(n) + C$

Résultats

	Barnes-Hut	FMM
Complexité	$O(n^2)$	$O(n \log(n)$
Erreur sur Terre (1 jour)	219.559km	219.559km
Par rapport à Horizons		112.49km

Hozizons : asteroïdes, relativité etc

Résultats

	FMM
Erreur sur Terre (1 jour) Par rapport au calcul naïf	500m
Erreur sur Terre (10 jour) Par rapport au calcul naïf	7500km

Annexes

We begin by defining the spherical harmonics of degree n and order m according to the formula

$$Y_n^m(\theta, \phi) = \sqrt{\frac{(n - |m|)!}{(n + |m|)!}} \cdot P_n^{|m|}(\cos\theta) e^{im\phi}. \tag{4}$$

Here, the special functions P_n^m are the associated Legendre functions, which can be defined by Rodrigues' formula

$$P_n^m(x) = (-1)^m (1 - x^2)^{m/2} \frac{d^m}{dx^m} P_n(x),$$

where $P_n(x)$ denotes the Legendre polynomial of degree n.

THEOREM 2.1 (Multipole expansion). *Suppose that N charges of strengths $q_1, q_2, \ldots, q_N$ are located at points $X_1, X_2, \ldots, X_N$ with spherical coordinates $(\rho_1, \alpha_1, \beta_1)$, $(\rho_2, \alpha_2, \beta_2), \ldots, (\rho_N, \alpha_N, \beta_N)$, respectively. Suppose further that the points $X_1, X_2, \ldots, X_N$ are located inside a sphere of radius a centered at the origin. Then, for any point $X = (r, \theta, \phi) \in \mathbb{R}^3$ with $r > a$, the potential $\Phi(X)$, generated by the charges $q_1, q_2, \ldots, q_N$, is given by the formula*

$$\Phi(X) = \sum_{n=0}^{\infty} \sum_{m=-n}^{n} \frac{M_n^m}{r^{n+1}} \cdot Y_n^m(\theta, \phi), \tag{5}$$

where

$$M_n^m = \sum_{i=1}^{N} q_i \cdot \rho_i^n \cdot Y_n^{-m}(\alpha_i, \beta_i). \tag{6}$$

Furthermore, for any $p \geq 1$,

$$\left| \Phi(X) - \sum_{n=0}^{p} \sum_{m=-n}^{n} \frac{M_n^m}{r^{n+1}} \cdot Y_n^m(\theta, \phi) \right| \leq \left(\frac{\sum_{i=1}^{N} |q_i|}{r - a} \right) \left(\frac{a}{r} \right)^{p+1}. \tag{7}$$

The preceding theorem describes an efficient representation of the far field due to a collection of sources. Within the FMM, it is also useful to be able to describe the field locally when the charges themselves are far away.

THEOREM 2.2 (Local expansion). *Suppose that N charges of strengths $q_1, q_2, \ldots, q_N$ are located at the points $X_1, X_2, \ldots, X_N$ in $\mathbb{R}^3$ with spherical coordinates $(\rho_1, \alpha_1, \beta_1)$, $(\rho_2, \alpha_2, \beta_2), \ldots, (\rho_N, \alpha_N, \beta_N)$, respectively. Suppose further that all the points $X_1, X_2, \ldots, X_N$ are located outside the sphere S_a of radius a centered at the origin. Then, for any point $X \in S_a$ with coordinates (r, θ, ϕ), the potential $\Phi(X)$ generated by the charges $q_1, q_2, \ldots, q_N$ is described by the local expansion*

$$\Phi(X) = \sum_{j=0}^{\infty} \sum_{k=-j}^{j} L_j^k \cdot Y_j^k(\theta, \phi) \cdot r^j, \tag{8}$$

where

$$L_j^k = \sum_{l=1}^{N} q_l \cdot \frac{Y_j^{-k}(\alpha_l, \beta_l)}{\rho_l^{j+1}}. \tag{9}$$

Furthermore, for any $p \geq 1$,

$$\left| \Phi(X) - \sum_{j=0}^{p} \sum_{k=-j}^{j} L_j^k \cdot Y_j^k(\theta, \phi) \cdot r^{j+1} \right| \leq \left(\frac{\sum_{i=1}^{N} |q_i|}{a - r} \right) \left(\frac{r}{a} \right)^{p+1}. \tag{10}$$

THEOREM 2.3 (Translation of a multipole expansion). *Suppose that N charges of strengths $q_1, q_2, \ldots, q_N$ are located inside the sphere D of radius a centered at $X_0 = (\rho, \alpha, \beta)$. Suppose further that for any point $X = (r, \theta, \phi) \in \mathbb{R}^3 \setminus D$, the potential due to these charges is given by the multipole expansion*

$$\Phi(X) = \sum_{n=0}^{\infty} \sum_{m=-n}^{n} \frac{O_n^m}{r'^{n+1}} \cdot Y_n^m(\theta', \phi'), \tag{11}$$

where (r', θ', ϕ') are the spherical coordinates of the vector $X - X_0$.

Then, for any point $X = (r, \theta, \phi)$ outside a sphere D_1 of radius $(a + \rho)$ centered at the origin,

$$\Phi(X) = \sum_{j=0}^{\infty} \sum_{k=-j}^{j} \frac{M_j^k}{r^{j+1}} \cdot Y_j^k(\theta, \phi), \tag{12}$$

where

$$M_j^k = \sum_{n=0}^{j} \sum_{m=-n}^{n} \frac{O_{j-n}^{k-m} \cdot i^{|k|-|m|-|k-m|} \cdot A_n^m \cdot A_{j-n}^{k-m} \cdot \rho^n \cdot Y_n^{-m}(\alpha, \beta)}{A_j^k}, \tag{13}$$

with A_n^m defined by the formula

$$A_n^m = \frac{(-1)^n}{\sqrt{(n-m)! \cdot (n+m)!}}. \tag{14}$$

Furthermore, for any $p \geq 1$,

$$\left| \Phi(X) - \sum_{j=0}^{p} \sum_{k=-j}^{j} \frac{M_j^k}{r^{j+1}} \cdot Y_j^k(\theta, \phi) \right| \leq \left(\frac{\sum_{i=1}^{N} |q_i|}{r - (a + \rho)} \right) \left(\frac{a + \rho}{r} \right)^{p+1}. \tag{15}$$

DEFINITION 2.1. Formula (13) defines a linear operator converting the multipole expansion coefficients $\{O_j^k\}$ into the multipole expansion coefficients $\{M_j^k\}$. This linear mapping will be denoted by $\mathcal{T}_{MM}$.

THEOREM 2.4 (Conversion of a multipole expansion to a local expansion). *Suppose that N charges of strengths $q_1, q_2, \ldots, q_N$ are located inside the sphere D_{X_0} of radius a centered at the point $X_0 = (\rho, \alpha, \beta)$, and that $\rho > (c+1)a$ for some $c > 1$. Then the corresponding multipole expansion (11) converges inside the sphere D_0 of radius a centered at the origin. Furthermore, for any point $X \in D_0$ with coordinates (r, θ, ϕ), the potential due to the charges $q_1, q_2, \ldots, q_N$ is described by the local expansion*

$$\Phi(X) = \sum_{j=0}^{\infty} \sum_{k=-j}^{j} L_j^k \cdot Y_j^k(\theta, \phi) \cdot r^j, \tag{16}$$

where

$$L_j^k = \sum_{n=0}^{\infty} \sum_{m=-n}^{n} \frac{O_n^m \cdot i^{|k-m|-|k|-|m|} \cdot A_n^m \cdot A_j^k \cdot Y_{j+n}^{m-k}(\alpha, \beta)}{(-1)^n A_{j+n}^{m-k} \cdot \rho^{j+n+1}}, \tag{17}$$

---

with A_n^m defined by (14). Furthermore, for any $p \geq 1$,

$$\left| \Phi(X) - \sum_{j=0}^{p} \sum_{k=-j}^{j} L_j^k \cdot Y_j^k(\theta, \phi) \cdot r^{j+1} \right| \leq \left(\frac{\sum_{i=1}^{N} |q_i|}{ca - a} \right) \left(\frac{1}{c} \right)^{p+1}. \tag{18}$$

DEFINITION 2.2. Formula (17) defines a linear operator converting the multipole expansion coefficients $\{O_j^k\}$ into the local expansion coefficients $\{L_j^k\}$. This linear mapping will be denoted by $\mathcal{T}_{ML}$.

THEOREM 2.5 (Translation of a local expansion). *Suppose that X_0, X are a pair of points in $\mathbb{R}^3$ with spherical coordinates (ρ, α, β), (r, θ, ϕ), respectively, and (r', θ', ϕ') are the spherical coordinates of the vector $X - X_0$ and p is a natural number. Let X_0 be the center of a pth-order local expansion with p finite; its expression at the point X is given by the formula*

$$\Phi(X) = \sum_{n=0}^{p} \sum_{m=-n}^{n} O_n^m \cdot Y_n^m(\theta', \phi') \cdot r'^n. \tag{19}$$

Then

$$\Phi(X) = \sum_{j=0}^{p} \sum_{k=-j}^{j} L_j^k \cdot Y_j^k(\theta, \phi) \cdot r^j, \tag{20}$$

everywhere in $\mathbb{R}^3$, with

$$L_j^k = \sum_{n=j}^{p} \sum_{m=-n}^{n} \frac{O_n^m \cdot i^{|m|-|m-k|-|k|} \cdot A_{n-j}^{m-k} \cdot A_j^k \cdot Y_{n-j}^{m-k}(\alpha, \beta) \cdot \rho^{n-j}}{(-1)^{n+j} \cdot A_n^m}, \tag{21}$$

and A_n^m are defined by (14).

DEFINITION 2.3. Formula (21) defines a linear operator converting the local expansion coefficients $\{O_n^m\}$ into the local expansion coefficients $\{L_n^m\}$. This linear mapping will be denoted by $\mathcal{T}_{LL}$.

DEFINITION 4.1. *List 1* of a childless box b, denoted by $L_1(b)$, is defined to be the set consisting of b and all childless boxes adjacent to b. If b is a parent box, its List 1 is empty.

DEFINITION 4.2. *List 2* of a box b, denoted by $L_2(b)$, is the set consisting of all children of the colleagues of b's parent that are well separated from b.

DEFINITION 4.3. *List 3* of a childless box b, denoted by $L_3(b)$, is the set consisting of all descendents of b's colleagues that are not adjacent to b, but whose parent boxes are adjacent to b. If b is a parent box, its list 3 is empty. Note that any box c in $L_3(b)$ is smaller than b and is separated from b by a distance not less than the side of c and not greater than the side of b.

DEFINITION 4.4. *List 4* of a box b, denoted by $L_4(b)$, consists of boxes c such that $b \in L_3(c)$; in other words, $c \in L_4(b)$ if and only if $b \in L_3(c)$. Note that all boxes in $L_4(b)$ are childless and are larger than b.

```c
typedef struct {
    double x, y, z;
    double vx, vy, vz;
    double ax, ay, az;
    double ax2, ay2, az2;
    double m;
} particle;

typedef struct {
    double x;
    double y;
    double z;
    double l;

    complex_double** local;
    complex_double** multipole;

    int parent;
    int child[8];
    int level;

    int first_particle;
    int n_particles;

    int* colleagues;
    int n_colleagues;

    int* L1;
    int n_L1;

    int* L2;
    int n_L2;

    int* L3;
    int n_L3;

    int* L4;
    int n_L4;
} box;

typedef struct {
    box* elements;
    int size;
    int capacity;

    int max_level;
    int** level_nodes;
    int* level_counts;
} octree;

typedef struct {
    particle* w;
    octree* o;
    double** a;
    double* fact;
} data;
```

```c
double* init_fact();

double factorial(int n, double* fact);

octree* create_octree(double simulation_size, int N);

void postorder(octree* o, void* param, int current, void (*f)(int, void*, octree*));

void preorder(octree* o, void* param, int current, void (*f)(int, void*, octree*));

void free_node(int x, void* param, octree* o);

void free_octree(octree* o);

void check_size(octree* o);

bool in_box(particle part, box* b);

void swap(particle* world, int i, int j);

int max(int x, int y);

int min(int x, int y);

void recursive_build(octree* o, particle* world, int b_index);

octree* init_octree(particle* world, int N, double simulation_size);

particle* init_world(int N, double simulation_size);

double** calculate_a_coefficients(double* fact);

data* init_data(int N, double simulation_size);

void free_data(data* d);

void counter_tab_incr(int x, void* param, octree* o);

void fill_levels(int x, void* param, octree* o);

void build_level_array(octree* o);

bool are_colleagues(box* b, box* c);

void build_colleagues(octree* o);

bool well_separated(box* b, box* c);

void build_L2(octree* o);
```

```
112  double double_comp(double x, double y);
113
114  bool are_adjacent(box* b, box* c);
115
116  bool is_parent(box* b);
117
118  void L1_count(int i, void* y, octree* o);
119
120  void L1_add(int i, void* y, octree* o);
121
122  void build_L1(octree* o);
123
124  void L3_L4_count(octree* o, box* b, box* x);
125
126  void L3_add(octree* o, box* b, box* x);
127
128  void build_L3_L4(octree* o);
129
130  void step_0(octree* o);
131
132  double associated_legendre(int m, int l, double x, double* fact);
133
134  complex_double harmonic(int m, int n, double x, double y, double z, double r, double* fact);
135
136  complex_double multipole_coef(int n, int m, particle* world, box* b, double* fact);
137
138  complex_double** calculate_multipole_child(box* b, particle* world, double* fact);
139
140  complex_double formula_13(int k, int j, box* b, double** a, double* fact, octree* o);
141
142  complex_double** TMM(box* b, octree* o, double** a, double* fact);
143
144  void upward_pass(int i, void* x, octree* o);
145
146  double dist(particle* a, particle* b);
147
148  //theorem 2.2
149  complex_double local_coef(int* tab, int size, int k, int j, particle* world, octree* o, double* fact);
150
151  void calculate_local(int* tab, int size, box* b, particle* world, octree* o, double* fact);
152
153  complex_double formula_17(int* tab, int size, int k, int j, box* b, octree* o, double** a, double* fact);
154
155  void TML(int* tab, int size, box* b, octree* o, double** a, double* fact);
156
157  complex_double formula_21(int k, int j, box* b, box* c, double** a, double* fact);
158
159  void TLL(box* b, box* c, double** a, double* fact);
```

```
91  void postorder(octree* o, void* param, int current, void (*f)(int, void*, octree*)) {
92      for (int i = 0 ; i < 8 ; ++i) {
93          if (o->elements[current].child[i] != -1) {
94              postorder(o, param, o->elements[current].child[i], *f);
95          }
96      }
97      (*f)(current, param, o);
98      return;
99  }
100
101 void preorder(octree* o, void* param, int current, void (*f)(int, void*, octree*)) {
102     (*f)(current, param, o);
103     for (int i = 0 ; i < 8 ; ++i) {
104         if (o->elements[current].child[i] != -1) {
105             preorder(o, param, o->elements[current].child[i], *f);
106         }
107     }
108     return;
109 }
```

```
161 void downward_pass(int i, void* param, octree* o);
162
163 void fmm(data* d);
164
165 void update_positions(data* d, int N, double dt);
166
167 void reset_data(data* d, int N, double simulation_size)
168
169 void write_data(data* d, int N, FILE* file);
```

```
739 void upward_pass(int i, void* x, octree* o) {
740     data* d = (data*)x;
741     particle* world = d->w;
742     double** a = d->a;
743     double* fact = d->fact;
744     box* b = &o->elements[i];
745     if (is_parent(b)) { // step 2
746         b->multipole = TMM(b, o, a, fact);
747     }
748     else { // step 1
749         b->multipole = calculate_multipole_child(b, world, fact);
750     }
751     return;
752 }
```

```
842  void downward_pass(int i, void* param, octree* o) {
843      data* d = (data*)param;
844      particle* world = d->w;
845      double** a = d->a;
846      double* fact = d->fact;
847      box* b = &o->elements[i];
848      if (i == 0) {
849          for (int j = 0 ; j < 8 ; ++j) {
850              if (b->child[j] != -1) {
851                  box* c = &o->elements[b->child[j]];
852                  c->local = (complex_double**)malloc((P+1)*sizeof(complex_double*));
853                  for (int k = 0 ; k < P + 1 ; ++k) {
854                      c->local[k] = (complex_double*)malloc((2*k+1)*sizeof(complex_double));
855                      for (int l = 0 ; l < 2*k+1 ; ++l) {
856                          c->local[k][l] = zero();
857                      }
858                  }
859              }
860          }
861      }
```

```
862     else {
863         //step 3 -> step 4 assuming tables exist
864         if (b->n_particles <= P*P) {
865             for (int j = 0 ; j < b->n_L4 ; ++j) {
866                 box* c = &o->elements[b->L4[j]];
867                 for (int k = 0 ; k < b->n_particles ; ++k) {
868                     particle* b_part = world + b->first_particle + k;
869                     for (int l = 0 ; l < c->n_particles ; ++l) {
870                         particle* c_part = world + c->first_particle + l;
871                         double dx = c_part->x - b_part->x;
872                         double dy = c_part->y - b_part->y;
873                         double dz = c_part->z - b_part->z;
874 
875                         double r2 = dx*dx + dy*dy + dz*dz + CUSHION*CUSHION;
876                         double invr = 1.0/sqrt(r2);
877                         double invr3 = invr*invr*invr;
878 
879                         b_part->ax += G * c_part->m * dx * invr3;
880                         b_part->ay += G * c_part->m * dy * invr3;
881                         b_part->az += G * c_part->m * dz * invr3;
882                     }
883                 }
884             }
885         }
886         else {
887             (void)calculate_local(b->L4, b->n_L4, b, world, o, fact);
888         }
889         (void)TML(b->L2, b->n_L2, b, o, a, fact); // step 4
890         if (is_parent(b)) { //step 5 and init children
891             for (int t = 0 ; t < 8 ; ++t) {
892                 if (b->child[t] != -1) {
893                     box* c = &o->elements[b->child[t]];
894                     c->local = (complex_double**)malloc((P+1) * sizeof(complex_double*));
895                     for (int j = 0 ; j < P + 1 ; ++j) {
896                         c->local[j] = (complex_double*)malloc((2*j+1) * sizeof(complex_double));
897                         for (int l = 0 ; l < 2*j+1 ; ++l) {
898                             c->local[j][l] = zero();
899                         }
900                     }
901                     TLL(b, c, a, fact); //shift to children
902                 }
903             }
904         }
```

```
905         else {
906             for (int l = 0 ; l < b->n_particles ; ++l) {
907                 particle* part = &world[b->first_particle + l];
908                 double x = part->x - b->x;
909                 double y = part->y - b->y;
910                 double z = part->z - b->z;
911                 double r = sqrt(x*x + y*y + z*z);
912                 //step 6
913                 complex_double** h = (complex_double**)malloc((P+2)*sizeof(complex_double*)); //P+2 because derivatives
914                 complex_double** hx = (complex_double**)malloc((P+1)*sizeof(complex_double*));
915                 complex_double** hy = (complex_double**)malloc((P+1)*sizeof(complex_double*));
916                 complex_double** hz = (complex_double**)malloc((P+1)*sizeof(complex_double*));
917                 h[P+1] = (complex_double*)malloc((2*(P+1)+1) * sizeof(complex_double));
918                 for (int n = 0 ; n < P + 1 ; ++n) {
919                     h[n] = (complex_double*)malloc((2*n+1) * sizeof(complex_double));
920                     hx[n] = (complex_double*)malloc((2*n+1) * sizeof(complex_double));
921                     hy[n] = (complex_double*)malloc((2*n+1) * sizeof(complex_double));
922                     hz[n] = (complex_double*)malloc((2*n+1) * sizeof(complex_double));
923                     for (int m = -n ; m < n + 1 ; ++m) {
924                         h[n][m+n] = harmonic(m, n, x, y, z, r, fact);
925                         h[n][m+n] = multiply_real(multiply_complex(h[n][m+n], b->local[n][m+n]), power(r, n+1));
926                         double alpha1 = sqrt(n-m+1);
927                         double alpha2 = sqrt(n+m+1);
928                         double alpha3 = sqrt(n+m);
929                         double alpha4 = sqrt(n-m);
930                         hz[n][m+n] = multiply_real(h[n][m+n], alpha1 * alpha2 / r);
931                         complex_double D_plus = zero();
932                         complex_double D_minus = zero();
933                         if (m+1+n <= 2*n) D_plus = multiply_real(h[n][m+1+n], alpha1 * alpha4 / (2*r));
934                         if (m-1+n >= 0) D_minus = multiply_real(h[n][m-1+n], -alpha2 * alpha3 / (2*r));
935                         hx[n][m+n] = add_complex(D_plus, D_minus);
936                         hy[n][m+n] = multiply_complex(complex_1(), add_complex(D_minus, multiply_real(D_plus, -1)));
937                         part->ax -= G*re(multiply_complex(hx[n][m+n], b->local[n][n+m]));
938                         part->ay -= G*re(multiply_complex(hy[n][m+n], b->local[n][n+m]));
939                         part->az -= G*re(multiply_complex(hz[n][m+n], b->local[n][n+m]));
940                     }
941                 }
```

```
942             //step 7
943             for (int j = 0 ; j < b->n_L1 ; ++j) {
944                 box* c = &o->elements[b->L1[j]];
945                 for (int k = 0 ; k < c->n_particles ; ++k) {
946                     particle* other = world + c->first_particle + k;
947                     double dx = other->x - part->x;
948                     double dy = other->y - part->y;
949                     double dz = other->z - part->z;
950 
951                     double r2 = dx*dx + dy*dy + dz*dz + CUSHION*CUSHION;
952                     double invr = 1.0/sqrt(r2);
953                     double invr3 = invr*invr*invr;
954 
955                     part->ax += G*other->m * dx * invr3;
956                     part->ay += G*other->m * dy * invr3;
957                     part->az += G*other->m * dz * invr3;
958                 }
959             }
```

```
960                 //step 8
961                 for (int j = 0 ; j < b->n_L3 ; ++j) {
962                     box* c = &o->elements[b->L3[j]];
963                     if (c->n_particles <= P*P) {
964                         for (int k = 0 ; k < c->n_particles ; ++k) {
965                             particle* other = world + c->first_particle + k;
966                             double dx = other->x - part->x;
967                             double dy = other->y - part->y;
968                             double dz = other->z - part->z;
969 
970                             double r2 = dx*dx + dy*dy + dz*dz + CUSHION*CUSHION;
971                             double invr = 1.0/sqrt(r2);
972                             double invr3 = invr*invr*invr;
973 
974                             part->ax += G*other->m * dx * invr3;
975                             part->ay += G*other->m * dy * invr3;
976                             part->az += G*other->m * dz * invr3;
977                         }
978                     }
979                     else {
980                         for (int n = 0 ; n < P + 1 ; ++n) {
981                             for (int m = -n ; m < n + 1 ; ++m) {
982                                 part->ax -= G*re(multiply_complex(hx[n][m+n], c->multipole[n][n+m]));
983                                 part->ay -= G*re(multiply_complex(hy[n][m+n], c->multipole[n][n+m]));
984                                 part->az -= G*re(multiply_complex(hz[n][m+n], c->multipole[n][n+m]));
985                             }
986                         }
987                     }
988                 }
989                 for (int n = 0 ; n < P + 1 ; ++n) {
990                     free(h[n]);
991                     free(hx[n]);
992                     free(hy[n]);
993                     free(hz[n]);
994                 }
995                 free(h[P+1]);
996                 free(h);
997                 free(hx);
998                 free(hy);
999                 free(hz);
1000             }
1001         }
1002     }
1003 }
```